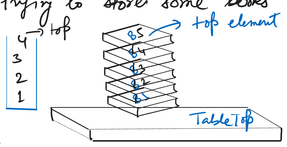
Continuation of linear data structures :→

Introduction to Stacks :→

It is a linear data structure which follows a very special property → LIFO → Last In First Out

Example : ① Imagine a table in your dining room where you are trying to store some plates / dishes after washing them.

② Or maybe you are trying to store some books on your study table.



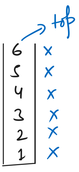
B1 → first to get in
B1 → Last to get out

BS → Last to get in
BS → First to get out

FILO

pop → 4, 3, 2, 1
push → 1, 2, 3, 4

Important functions of a stack :→

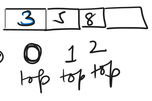


1. Adding Elements (push) ⇒ 1, 2, 3, 4, 5, 6
2. Removing Elements (pop) ⇒ 6, 5, 4, 3, 2, 1
3. Size( ) → check the size of stack
4. empty( ) → To check whether the stack is empty → Stack Underflow
5. full( ) → To check whether the stack is full → Stack Overflow
6. display ( ) → To display the elements
7. peek ( ) → To view the top element.

* Creating a stack with the help of an array :→

int top
index

top
-1 → empty

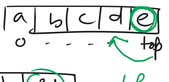


MAX_SIZE = 10

arr[top] = 3
arr[0] = 3

push (3) ⇒ top ++

if (top == MS -1) 5
10 -1 8
(top == 9)
(can't add) Stack overflow



top --

Case 1 : Empty	Case 2 : Single Element	Case 3 : Multiple elements
top -1 s = -1+1 = 0	x 0 top s = 0+1 = 1	a b c d 0 1 2 3 top s = 3+1 = 4

int size = top + 1;

Valid Parentheses :→

(LIFO) → Reverse (Stack)

String s1 (Characters) {E{ }[]} 0 1 2 3 4 5 true

String s2 {{(){}} false

{[ ]} ↳)[

[ ) mismatch : return false.

* If the character is an open bracket, push it into the stack. → 0, 1, 2, 3, 4, 5
  if s[i] == (or { or [
  stpush (s[i])

* If st.top() == (&& s[i] !=)
  st.top() == [&& s[i] !=]
  st.top() == { && s[i] != }
  return false;

* if it matches then pop the top of the stack.
* return st.empty()

"{ (["
" {]) "

st.empty()
↳ false;

stack is empty()

return false.